Trabalho 2 — Árvores Binárias

Árvores binárias de busca, árvores AVL e análise de complexidade

Instituto Federal do Espírito Santo — Campus Serra
Técnicas de Programação Avançada
Prof. Victorio Albani de Carvalho

Integrantes do grupo

Lucas Ramos Pianissola
Ryan Lima Peçanha
Davi Alvarenga

Repositório do projeto
https://github.com/ryanlimap/trabalho_tpa

Relatório completo: seção 1 (desenvolvimento), 2 (análise matemática da árvore binária), 3 (análise empírica da árvore binária), 4 (análise da árvore AVL), 5 (desempenho da AVL comparado ao da árvore binária comum) e 6 (árvores binárias na biblioteca padrão de Java).

1. Desenvolvimento da biblioteca de árvores e do programa de teste

1.1 Atuação de cada componente do grupo

O quadro abaixo descreve a participação de cada integrante nas **oito etapas** da especificação. Como no trabalho anterior, o trabalho foi dividido por módulo, mas as decisões de projeto, a revisão do código e a revisão final do relatório foram discutidas pelos três — e qualquer integrante pode responder por qualquer seção. A numeração das etapas segue a da especificação: 1 e 2 são a biblioteca de árvores e a adaptação do aplicativo; 3 é a descrição do desenvolvimento; 4 e 5 são a análise matemática e a análise empírica da árvore binária; 6 é a implementação da AVL; 7 é a avaliação da AVL; e 8 é a pesquisa sobre a biblioteca padrão de Java.

Integrante	Etapas	Atuação
Lucas Ramos Pianissola	1, 4 e 6	**Etapa 1** — a biblioteca de árvores: classes `NoArvore` e `ArvoreBinaria` (especializando `ArvoreBinariaBase`), com os métodos `adicionar`, `pesquisar`, `remover`, `quantidadeNos`, `altura`, `caminharEmNivel` e `caminharEmOrdem`. **Etapa 6** — classe `ArvoreAVL`, com as quatro rotações e o rebalanceamento da inserção e da remoção. Suítes de testes `app.TesteArvore` e `app.TesteAVL`. **Etapa 4** — análise matemática da árvore binária e da AVL (**redação das seções 2 e 4**), que decorre diretamente do código que implementou.
Ryan Lima Peçanha	2, 3 e 8	**Etapa 2** — adaptação do aplicativo: o menu inicial passou a oferecer lista não-ordenada, lista ordenada, árvore binária de busca ou AVL, mantendo o restante do programa inalterado graças à interface `IColecao`; correção da alteração de contato para impedir telefones duplicados. **Etapa 8** — pesquisa sobre árvores binárias na biblioteca padrão de Java e **redação da seção 6**. **Etapa 3** — **redação da seção 1** deste relatório (descrição do desenvolvimento, uso de ferramentas de IA e repositório) e organização do repositório no GitHub.
Davi Alvarenga	5 e 7	Infraestrutura das medições: adaptação do `GeradorArquivo` para os três modos de geração, `Benchmark` e `ContagemOperacoes` (incluindo o `ComparadorContador`). **Etapa 5** — execução de todas as rodadas de medição da árvore binária, gráficos e tabelas, e **redação da seção 3**. **Etapa 7** — as mesmas medições com a AVL e **redação da seção 5** (comparação AVL × árvore comum).

1.2 Como as ferramentas de IA foram utilizadas

Seguimos a mesma prática do trabalho anterior, usando uma ferramenta de IA generativa (Claude, da Anthropic) como apoio em frentes bem delimitadas.

a) Aproveitamento do material da disciplina. Fornecemos à ferramenta os slides de árvores da disciplina, os exemplos em C disponibilizados pelo professor (`arvBin.c`) e a classe `ArvoreBinariaBase` do repositório da disciplina. A ferramenta nos ajudou a transpor para Java, com *Generics* e `Comparator`, a lógica de inserção, busca e remoção vista em aula — inclusive a estratégia de remoção de nó com dois filhos por **predecessor** (o maior elemento da subárvore esquerda), que é a mesma usada no exemplo em C.

b) Discussão das decisões de implementação. A principal delas foi **escrever todos os percursos de forma iterativa**, e não recursiva. Os exemplos em C são recursivos, o que é natural em árvores; mas como este trabalho exige inserir centenas de milhares de elementos em uma árvore **degenerada** (altura n−1), a recursão estouraria a pilha da JVM com `StackOverflowError`. Discutimos a alternativa com a ferramenta, implementamos a versão iterativa e confirmamos com um teste que monta uma árvore degenerada de 100.000 nós e a percorre inteira sem estourar a pilha.

A AVL, por outro lado, é recursiva — e essa diferença é deliberada. A inserção e a remoção da AVL precisam executar, *na volta* de cada nível, a atualização da altura e o rebalanceamento, o que a recursão oferece de graça: cada chamada devolve a nova raiz da sua subárvore, no estilo das funções `inserirV2` e `removerV2` vistas em aula. Fazer isso de forma iterativa exigiria manter uma pilha explícita do caminho percorrido. O risco de estouro de pilha que nos obrigou a ser iterativos na árvore comum simplesmente não existe aqui: a AVL **garante** altura O(log n), que para 200.000 elementos são cerca de 25 níveis de recursão no limite teórico (e 17 medidos), contra os 199.999 da árvore degenerada.

c) Infraestrutura de medição e relatório. Os programas geradores de arquivos, as classes de medição e os scripts Python que produzem os gráficos e este PDF foram escritos com apoio da ferramenta, a partir dos dados das nossas execuções. Todos os números apresentados são medições nossas.

d) Revisão de texto. A redação final foi revisada com apoio da ferramenta, para clareza e correção da linguagem.

1.3 Repositório e organização do código

Todo o código está disponível em **https://github.com/ryanlimap/trabalho_tpa**, com um `README.md` descrevendo a organização das pastas e como compilar e executar o projeto.

Pasta / arquivo	Conteúdo
src/colecao/IColecao.java	Interface comum a listas e árvores (do professor).
src/colecao/ListaEncadeada.java	A biblioteca de listas do Trabalho 1, reaproveitada sem alterações.
src/arvorebinaria/ArvoreBinariaBase.java	Classe abstrata fornecida pelo professor, preservada como recebida.
src/arvorebinaria/NoArvore.java	Nó da árvore: valor do tipo T e as referências esquerda e direita.
src/arvorebinaria/ArvoreBinaria.java	**A biblioteca de árvores**: especializa `ArvoreBinariaBase`. Não imprime nada.
src/arvorebinaria/ArvoreAVL.java	**A árvore AVL**: herda de `ArvoreBinaria` e sobrescreve `adicionar` e `remover` para rebalancear.
src/app/Programa.java	A agenda: pergunta qual estrutura usar (lista não-ordenada, lista ordenada, árvore binária ou árvore AVL).
src/app/GeradorArquivo.java	Gera os arquivos de entrada em ordem crescente, balanceada ou aleatória.
src/app/Benchmark.java	Uma rodada de medição de tempo; imprime uma linha CSV.
src/app/ContagemOperacoes.java	Conta comparações em vez de medir tempo.
src/app/TesteArvore.java	Testes automatizados da árvore binária (30 verificações).

Pasta / arquivo	Conteúdo
src/app/TesteAVL.java	Testes da AVL: as quatro rotações, entrada crescente, remoções e 20.000 operações aleatórias verificando a propriedade AVL.
analise/ e dados/	Scripts dos gráficos e deste PDF; CSVs com as medições.

O que mudou no aplicativo. Como a `ArvoreBinariaBase` implementa `IColecao`, a única alteração necessária no programa do Trabalho 1 foi a pergunta inicial e a criação do objeto correspondente. Todo o resto — menu, carga de arquivo, buscas, remoção, alteração e medição de tempos — continua funcionando sem modificação, porque conversa apenas com a interface. Esse é exatamente o ganho de projeto que a interface oferece.

Listagem 1 — a única mudança estrutural no app — src/app/Programa.java, linhas 33-55

```
 33          int estrutura = perguntarEstrutura();
 34          switch (estrutura) {
 35              case 2 -> {
 36                  colecaoPorNome = new ListaEncadeada<>(new ComparadorContatoPorNome(), true);
 37                  colecaoPorTelefone = new ListaEncadeada<>(new ComparadorContatoPorTelefone(), true
);
 38                  nomeEstrutura = "listas ordenadas";
 39              }
 40              case 3 -> {
 41                  colecaoPorNome = new ArvoreBinaria<>(new ComparadorContatoPorNome());
 42                  colecaoPorTelefone = new ArvoreBinaria<>(new ComparadorContatoPorTelefone());
 43                  nomeEstrutura = "arvores binarias de busca";
 44              }
 45              case 4 -> {
 46                  colecaoPorNome = new ArvoreAVL<>(new ComparadorContatoPorNome());
 47                  colecaoPorTelefone = new ArvoreAVL<>(new ComparadorContatoPorTelefone());
 48                  nomeEstrutura = "arvores AVL";
 49              }
 50              default -> {
 51                  colecaoPorNome = new ListaEncadeada<>(new ComparadorContatoPorNome(), false);
 52                  colecaoPorTelefone = new ListaEncadeada<>(new ComparadorContatoPorTelefone(), fals
e);
 53                  nomeEstrutura = "listas nao-ordenadas";
 54              }
 55          }
```

2. Análise matemática de complexidade

Nesta seção analisamos os métodos `adicionar`, `pesquisar`, `remover` e `quantidadeNos` da classe `ArvoreBinaria`. A numeração das linhas é a numeração real do arquivo `src/arvorebinaria/ArvoreBinaria.java`.

Premissas. (i) *n* é a quantidade de nós da árvore e *h* a sua altura; (ii) atribuições, comparações, criação de um nó e desvios são operações de custo constante; (iii) a chamada `comparador.compare(...)` é O(1); (iv) analisamos o **pior caso**.

2.1 O ponto central: a topologia manda na complexidade

Diferentemente da lista encadeada, em uma árvore binária o custo dos quatro métodos analisados nesta seção não depende diretamente de *n*, e sim da **altura** da árvore — porque `adicionar`, `pesquisar` e `remover` seguem um **único caminho da raiz até um nó**, e nunca visitam a árvore inteira. (Os caminhamentos e o método `altura`, esses sim, visitam todos os nós e são O(n) em qualquer topologia.) E a altura, para um mesmo *n*, depende da **ordem em que os elementos foram inseridos**. Como visto em aula, vale a relação:

```
                        n - 1  >=  h  >=  piso(log2 n)

h = n - 1           arvore COMPLETAMENTE DEGENERADA: cada no tem um unico
                    filho; a arvore vira uma lista encadeada.
                    Ocorre quando os elementos sao inseridos em ordem
                    crescente (ou decrescente).

h = piso(log2 n)    arvore PERFEITAMENTE BALANCEADA: todos os niveis
                    completos, exceto possivelmente o ultimo (um nivel d
                    comporta ate 2^d nos). Para n = 2^k - 1 todos os niveis
                    ficam cheios; para os demais n, o ultimo nivel fica
                    parcialmente preenchido e a altura ainda e piso(log2 n).
                    Ocorre quando se insere sempre o elemento central do
                    intervalo restante.
```

Portanto, **toda operação que custa O(h) tem dois piores casos a declarar**: O(log n) se a árvore estiver perfeitamente balanceada e O(n) se estiver degenerada. É essa diferença que a seção 3 mede.

2.2 Método adicionar

Listagem 2 — adicionar — src/arvorebinaria/ArvoreBinaria.java, linhas 39-68

```
39      @Override
40      public boolean adicionar(T novoValor) {
41          if (novoValor == null) {
42              return false;
43          }
44          NoArvore<T> novo = new NoArvore<>(novoValor);
45          if (this.raiz == null) {
46              this.raiz = novo;
47              this.quant++;
48              return true;
49          }
50          NoArvore<T> atual = this.raiz;
51          while (true) {
52              if (comparador.compare(atual.getValor(), novoValor) > 0) {
53                  if (atual.getEsq() == null) {
54                      atual.setEsq(novo);
55                      break;
56                  }
57                  atual = atual.getEsq();
58              } else {
59                  if (atual.getDir() == null) {
60                      atual.setDir(novo);
61                      break;
62                  }
63                  atual = atual.getDir();
64              }
65          }
66          this.quant++;
67          return true;
68      }
```

As linhas 41-43 rejeitam `null` e a linha 44 cria o nó: custo constante. As linhas 45-49 tratam a árvore vazia, em que o novo elemento vira a raiz: também constante. O que define a complexidade é o laço das linhas 51-65: partindo da raiz, cada iteração faz **uma comparação** (linha 52) e desce um nível — para a esquerda, se o valor do nó atual for maior que o novo (linhas 53-57), ou para a direita, caso contrário (linhas 58-64). O laço termina quando encontra a posição livre onde o novo nó é pendurado.

Note que elementos de chave **igual** caem no `else` e são inseridos à direita — a mesma convenção do exemplo em C visto em aula.

O laço desce no máximo *h* níveis, então **T(n) = $c_1 \cdot h + c_2$**, ou seja, **O(h)**. Traduzindo para *n*:

Topologia	Altura	Custo de adicionar	Quando ocorre
Perfeitamente balanceada	piso($\log_2$ n)	**O(log n)**	Inserção na ordem “do meio para fora”, como no nosso arquivo balanceado.
Degenerada	n – 1	**O(n)**	Inserção em ordem crescente ou decrescente das chaves — o caso do nosso arquivo em ordem crescente.

Consequência: custo de montar a árvore com n elementos. Inserindo os *n* contatos um a um:

```
Arvore balanceada:  cada insercao custa O(log i) para i elementos ja
                    inseridos. Somando:  log1 + log2 + ... + logn  =
                    log(n!), que e  O(n log n).

Arvore degenerada:  a i-esima insercao percorre os i-1 nos ja inseridos
                    (todos estao em uma unica "espinha"). Somando:
                    0 + 1 + ... + (n-1) = n(n-1)/2  ->  O(n^2).
                    E EXATAMENTE a mesma soma da lista ordenada do
                    Trabalho 1 - porque a estrutura e, de fato, a mesma.
```

2.3 Método pesquisar

Listagem 3 — pesquisar — src/arvorebinaria/ArvoreBinaria.java, linhas 74-85

```
74      @Override
75      public T pesquisar(T valor) {
76          NoArvore<T> atual = this.raiz;
77          while (atual != null) {
78              int cmp = comparador.compare(atual.getValor(), valor);
79              if (cmp == 0) {
80                  return atual.getValor();
81              }
82              atual = (cmp > 0) ? atual.getEsq() : atual.getDir();
83          }
84          return null;
85      }
```

O método segue um único caminho a partir da raiz: a cada nó faz uma comparação (linha 78) e, pelo sinal, decide se devolve o valor (linhas 79-81), desce à esquerda ou desce à direita (linha 82). Se chega a `null`, o elemento não existe (linha 84).

É aqui que está a vantagem potencial da árvore sobre a lista: cada comparação **descarta uma subárvore inteira** — a busca binária que a lista encadeada não permitia, porque agora existe acesso direto aos dois filhos de cada nó. Note, porém, que o tamanho do que se descarta depende da topologia: na árvore **balanceada** cada comparação elimina cerca de metade dos nós restantes, enquanto na **degenerada** a subárvore descartada está sempre vazia e a comparação elimina apenas o próprio nó — exatamente como em uma lista.

Pior caso: O(h), que ocorre quando o elemento procurado está em uma **folha de profundidade máxima** (ou quando não existe e a busca só termina ao chegar a `null` no nível mais profundo). Em números: **O(log n)** na árvore perfeitamente balanceada e **O(n)** na degenerada — nesta última a árvore é uma lista, e a busca volta a ser linear.

Vale registrar o número exato de comparações desse pior caso, porque ele será conferido na seção 3.2.3: buscar uma folha que está no nível *h* exige visitar os *h* nós do caminho mais a própria folha, ou seja, **h + 1 comparações**. Em uma árvore degenerada construída a partir de chaves crescentes, h = n − 1 e a busca faz exatamente **n comparações**.

2.4 Método remover

Listagem 4 — remover — src/arvorebinaria/ArvoreBinaria.java, linhas 91-132

```
 91      @Override
 92      public boolean remover(T valor) {
 93          NoArvore<T> pai = null;
 94          NoArvore<T> atual = this.raiz;
 95          while (atual != null) {
 96              int cmp = comparador.compare(atual.getValor(), valor);
 97              if (cmp == 0) {
 98                  break;
 99              }
100              pai = atual;
101              atual = (cmp > 0) ? atual.getEsq() : atual.getDir();
102          }
103          if (atual == null) {
104              return false;
105          }
106          // Caso 3: dois filhos. Seguindo a mesma estratégia vista em aula, o nó
107          // recebe o valor do seu PREDECESSOR (o maior valor da subárvore
108          // esquerda) e passa-se a remover o nó do predecessor, que por ser o
109          // mais à direita da subárvore esquerda tem no máximo um filho.
110          if (atual.getEsq() != null && atual.getDir() != null) {
111              NoArvore<T> paiPredecessor = atual;
112              NoArvore<T> predecessor = atual.getEsq();
113              while (predecessor.getDir() != null) {
114                  paiPredecessor = predecessor;
115                  predecessor = predecessor.getDir();
116              }
117              atual.setValor(predecessor.getValor());
118              pai = paiPredecessor;
119              atual = predecessor;
120          }
121          // Casos 1 e 2: folha (filho == null) ou um único filho.
122          NoArvore<T> filho = (atual.getEsq() != null) ? atual.getEsq() : atual.getDir();
123          if (pai == null) {
124              this.raiz = filho;
125          } else if (pai.getEsq() == atual) {
126              pai.setEsq(filho);
127          } else {
128              pai.setDir(filho);
129          }
130          this.quant--;
131          return true;
132      }
```

A remoção tem duas fases. A **primeira** (linhas 93-105) é uma busca igual à do `pesquisar`, guardando também o nó **pai**, necessário para refazer a ligação: custo **O(h)**.

A **segunda** fase trata os três casos clássicos:

Caso	Linhas	O que acontece	Custo
Nó com dois filhos	110-120	O nó recebe o valor do seu **predecessor** (o maior da subárvore esquerda, encontrado descendo sempre à direita) e passa-se a remover o nó do predecessor, que tem no máximo um filho.	O(h)
Nó folha	122-129	`filho` fica `null` e o pai passa a apontar para `null`.	O(1)
Nó com um filho	122-129	O pai passa a apontar diretamente para o único filho do nó removido.	O(1)

A busca pelo predecessor (laço das linhas 113-116) desce sempre pela direita a partir do filho esquerdo do nó removido. É importante notar que ela **não** acrescenta outros *h* níveis ao custo: a descida até o nó e a descida até o predecessor formam um **único caminho que parte da raiz**, de modo que o total de nós visitados é no máximo **h + 1**. Somando as duas fases, **T(n) = c·h + c'**, ou seja, **O(h)** — novamente **O(log n)** em árvore balanceada e **O(n)** em árvore degenerada.

Quando o pior caso ocorre. Na árvore **balanceada**, ao remover um nó cujo predecessor está no último nível — ou a própria folha de profundidade máxima —, percorrendo os h+1 nós do caminho. Na árvore **degenerada** gerada por chaves crescentes há uma particularidade que vale mencionar na

entrevista: como todo nó tem apenas filho direito, **nenhum nó tem dois filhos** e o caso 3 nunca chega a ser executado; o pior caso é simplesmente remover o último nó da espinha, que custa a varredura completa — O(n).

Vale observar o contraste com a lista do Trabalho 1: lá, a religação em si era O(1) e o custo todo estava em **encontrar** o nó. Aqui vale o mesmo princípio, só que encontrar passou a custar O(h) em vez de O(n) — quando a árvore é balanceada.

2.5 Método quantidadeNos

Listagem 5 — quantidadeNos — src/arvorebinaria/ArvoreBinaria.java, linhas 138-141

```
138      @Override
139      public int quantidadeNos() {
140          return this.quant;
141      }
```

Como na lista, mantemos um contador atualizado na inserção (linhas 47 e 66) e na remoção (linha 130). O método apenas o devolve: **T(n) = 1**, portanto **O(1)**, **independentemente da topologia**. Se o contador não existisse, seria preciso percorrer toda a árvore e o método seria O(n).

2.6 Resumo e comparação com as listas do Trabalho 1

Método	Árvore balanceada	Árvore degenerada	Lista ordenada	Lista não-ordenada
adicionar	O(log n)	O(n)	O(n)	O(1)
pesquisar	O(log n)	O(n)	O(n)	O(n)
remover	O(log n)	O(n)	O(n)	O(n)
quantidadeNos	O(1)	O(1)	O(1)	O(1)
montar n elementos	O(n log n)	$O(n^2)$	$O(n^2)$	O(n)

A leitura da tabela é o resumo do trabalho: **a árvore binária só é melhor que a lista enquanto a sua topologia for favorável**. Perfeitamente balanceada, ela transforma O(n) em O(log n) nas três operações principais. Degenerada, ela **é** uma lista encadeada — com o agravante de gastar um ponteiro a mais por nó, já que o campo da esquerda fica sempre nulo.

É por isso que existem as árvores balanceadas (AVL), objeto das próximas seções deste trabalho: elas garantem h = O(log n) **independentemente da ordem de inserção**, eliminando o pior caso que medimos aqui.

3. Análise empírica de complexidade

3.1 Implementação: geração dos arquivos e definição do pior caso

Como geramos os arquivos de entrada. Adaptamos o gerador do Trabalho 1 (classe `app.GeradorArquivo`) para produzir os contatos em três ordens diferentes, controlando a topologia da árvore resultante. Cada contato tem um identificador *i*; o telefone é a **concatenação** do prefixo `"27"` com o número `900000000 + i` e o nome é `"Contato "` seguido de *i* com zeros à esquerda. Para *i* = 5, por exemplo, o registro gerado é `Contato 0000005;27900000005` — o nome é derivado do mesmo identificador **para que a ordem alfabética dos nomes seja idêntica à ordem numérica dos telefones**, de modo que as duas árvores (a indexada por nome e a indexada por telefone) tenham a mesma topologia controlada.

Os dois modos de geração usados nos testes

```
CRESCENTE  (arquivo cres_N.txt)  ->  arvore DEGENERADA
   Emite os identificadores 0, 1, 2, 3, ... em ordem.
   Cada novo elemento e maior que todos os ja inseridos, entao desce
   sempre para a direita e vira filho direito do ultimo. A arvore nao
   tem nenhum filho a esquerda: altura = n - 1.

BALANCEADO (arquivo bal_N.txt)   ->  arvore PERFEITAMENTE BALANCEADA
   Emite o elemento do MEIO do intervalo [0, n-1]; depois o meio de cada
   metade; depois o meio de cada quarto, e assim por diante (percurso em
   largura sobre os intervalos).
   Exemplo com n = 15:  7 | 3 11 | 1 5 9 13 | 0 2 4 6 8 10 12 14
   Inserindo nessa ordem, cada elemento cai exatamente no lugar que
   completa o nivel: altura = piso(log2 n).
```

Como definimos o pior caso de busca. A especificação pede uma busca pela folha mais distante da raiz, considerando a indexação por telefone. Em vez de deduzir qual elemento seria esse em cada arquivo, a própria biblioteca responde: implementamos na `ArvoreBinaria` o método `valorDaFolhaMaisProfunda()`, que faz um percurso em largura e devolve o valor do **último nó visitado** — que está necessariamente no nível mais profundo, e portanto é uma folha de profundidade máxima. Essa é, por definição, a busca com sucesso mais cara da árvore.

Listagem 6 — identificação do pior caso de busca — src/arvorebinaria/ArvoreBinaria.java, linhas 238-256

```
238     public T valorDaFolhaMaisProfunda() {
239         if (this.raiz == null) {
240             return null;
241         }
242         NoArvore<T> ultimo = this.raiz;
243         Deque<NoArvore<T>> fila = new ArrayDeque<>();
244         fila.add(this.raiz);
245         while (!fila.isEmpty()) {
246             NoArvore<T> no = fila.poll();
247             ultimo = no;
248             if (no.getEsq() != null) {
249                 fila.add(no.getEsq());
250             }
251             if (no.getDir() != null) {
252                 fila.add(no.getDir());
253             }
254         }
255         return ultimo.getValor();
256     }
```

O método é chamado sobre a **árvore indexada por telefone**, como a especificação determina, e o contato devolvido por ele é o **mesmo** usado nas três medições seguintes: a busca por telefone, a busca por nome e a remoção por telefone.

Sobre usar uma classe de medição separada. As medições são feitas pela classe `app.Benchmark`, e não digitando as opções no menu do `app.Programa`. Ela executa exatamente a mesma sequência de operações que o menu executaria, sobre **as mesmas bibliotecas** e através da **mesma interface** `IColecao` — a única diferença é que ela dispensa a digitação e imprime os tempos já em formato CSV, o que torna viável repetir dezenas de execuções de forma automatizada e reprodutível.

Metodologia das medições. Para cada arquivo, a classe `app.Benchmark` (i) lê o arquivo e monta as duas árvores, cronometrando o tempo total; (ii) identifica a folha mais profunda; (iii) pesquisa esse contato pelo telefone; (iv) pesquisa o mesmo contato pelo nome; (v) remove-o pelo telefone. Os tempos são medidos com `System.nanoTime()` e cada medição roda em uma **JVM nova**. O tempo de **montagem** é a mediana de 3 execuções completas. Já as três **operações** (busca por telefone, busca por nome e remoção) são repetidas **5 vezes dentro de cada execução**, e reportamos a mediana dessas repetições — o contato removido é reinserido após cada medição, de modo que todas partem do mesmo estado. Isso foi necessário porque essas operações duram de microssegundos a poucos milissegundos, faixa em que uma medida isolada é dominada por ruído do sistema operacional e do coletor de lixo. Ao todo, cada ponto das tabelas de operação resume 15 medições.

Uma exceção a registrar. A **lista ordenada**, usada apenas como termo de comparação na seção 3.2.4, foi executada **uma única vez** por tamanho, e não 3, porque a montagem dessa estrutura leva vários minutos nos tamanhos maiores. Por isso os seus tempos devem ser lidos como indicativos: o valor de 8,62 ms na busca com 200.000 elementos, por exemplo, é alto demais para a tendência da própria série e provavelmente reflete ruído de uma medida isolada. Isso não afeta nenhuma conclusão do trabalho, que se apoia nas árvores e na contagem de comparações.

Além do tempo, a classe `app.ContagemOperacoes` conta o **número de comparações** executadas. Como no Trabalho 1, essa medida independe de máquina, JIT e cache, e é a evidência mais limpa da ordem de crescimento.

Ambiente de execução: AMD Ryzen 5 5600G (6 núcleos, 16 MB de cache L3), 32 GB de RAM DDR4 3200 MHz, Windows 11, OpenJDK 21 (64 bits). Todas as medições apresentadas nas seções 3 e 5 foram coletadas nessa mesma máquina, sem outros programas concorrendo pela CPU.

Tamanhos usados: 25.000, 50.000, 100.000, 200.000 contatos, os mesmos do Trabalho 1, em dois arquivos cada (crescente e balanceado).

3.2 Análise de complexidade

Altura das árvores geradas. Antes dos tempos, a medida que explica todo o resto:

n	Altura — balanceada	piso($\log_2 n$)	Altura — degenerada	n − 1
25.000	14	14	24.999	24.999
50.000	15	15	49.999	49.999
100.000	16	16	99.999	99.999
200.000	17	17	199.999	199.999

Os geradores fizeram exatamente o que deveriam: a árvore do arquivo balanceado atingiu a **altura mínima teórica** piso($\log_2 n$) em todos os tamanhos, e a do arquivo crescente atingiu a **altura máxima** n−1. Com 200.000 contatos, uma árvore tem altura 17 e a outra 199.999: é essa diferença de quatro ordens de grandeza que aparece em todos os gráficos a seguir.

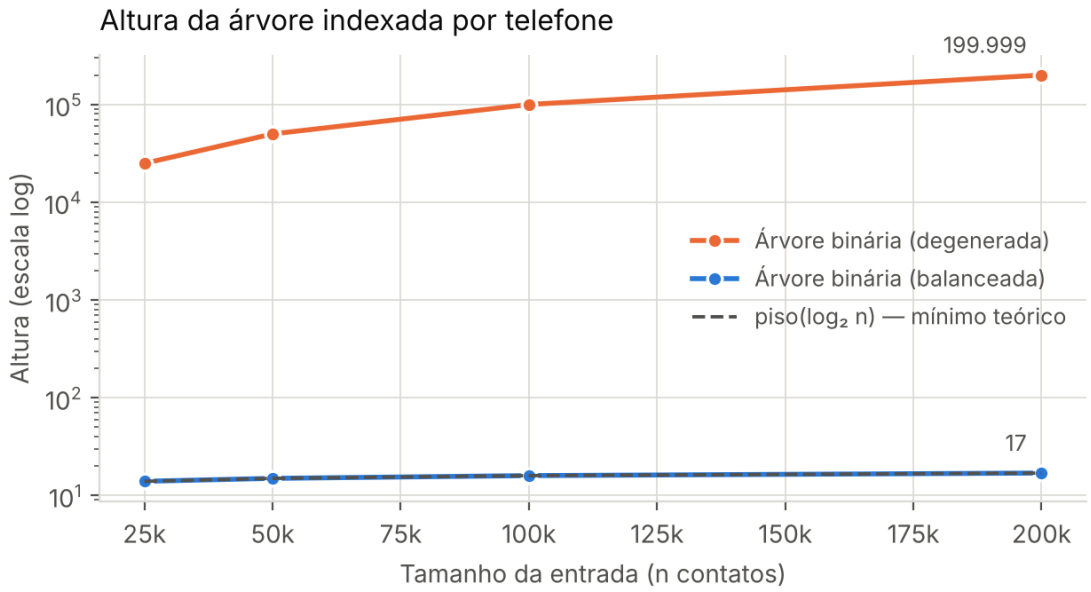


Figura 1 — Altura das árvores (escala logarítmica). A curva da árvore balanceada coincide com o mínimo teórico piso($\log_2 n$).

3.2.1 Tempo de leitura do arquivo e montagem das árvores

n	Montagem (ms) árvore balanceada	razão	Montagem (ms) árvore degenerada	razão
25.000	46,04	—	4.888,70	—
50.000	61,95	1,35×	27.967,14	5,72×
100.000	101,38	1,64×	132.331,62	4,73×
200.000	168,45	1,66×	464.520,49	3,51×

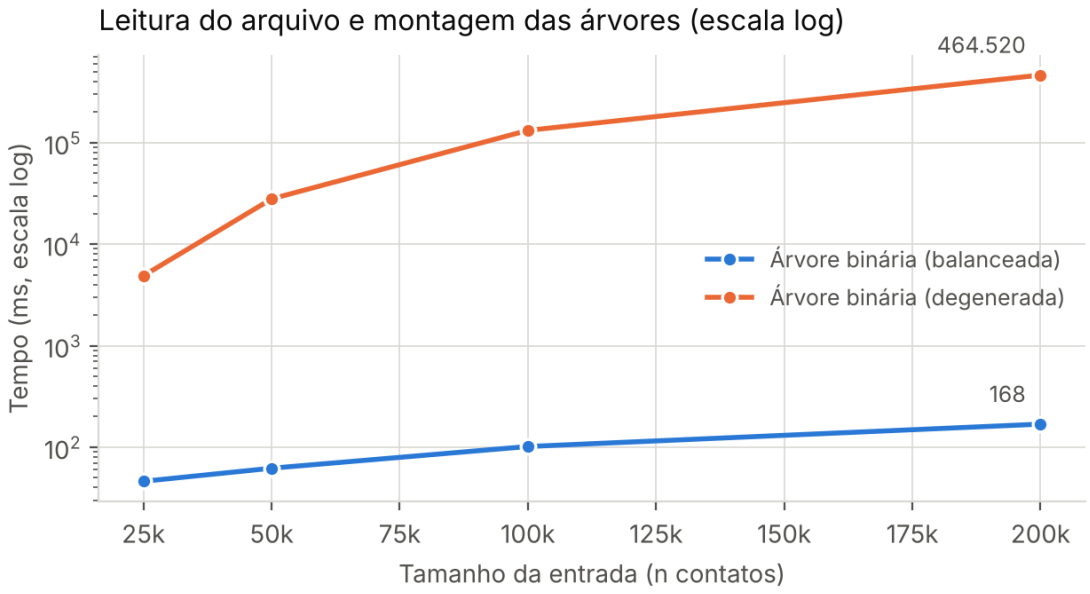


Figura 2 — Montagem das árvores (escala logarítmica).

Interpretação. Na árvore **balanceada**, dobrar *n* multiplica o tempo por valores entre 1,35×, 1,64× e 1,66× — compatível com o O(n log n) previsto na seção 2.2. O fator teórico ao dobrar *n* é 2·log(2n)/log(n), que para esses tamanhos vale cerca de 2,1; as razões medidas ficam abaixo disso porque parte do tempo não cresce com *n* da mesma forma: a compilação JIT dos métodos acontece nas primeiras milhares de inserções e a abertura do arquivo tem custo próprio. Note que a inicialização da JVM **não** entra nessa conta — o cronômetro `System.nanoTime()` só começa dentro do programa, depois que a máquina virtual já subiu.

Na árvore **degenerada**, as razões são 5,72×, 4,73× e 3,51× a cada duplicação de *n* — a assinatura de **O(n²)**, exatamente o previsto na seção 2.2. Com 200.000 contatos a montagem degenerada levou **7,7 minutos**, contra **168 ms** da balanceada: cerca de **2.758 vezes mais**, com o **mesmo número de elementos e o mesmo código**. A única diferença entre as duas execuções é a ordem em que os dados chegaram.

As razões medidas no relógio oscilam em torno de 4, ora acima, ora abaixo. Essa dispersão vem do **custo de cada operação**, não da quantidade delas: percorrer uma estrutura encadeada de centenas de milhares de nós espalhados pelo heap gera um número de faltas de cache que varia com o tamanho da estrutura e com o estado da máquina — o mesmo efeito de hierarquia de memória discutido no Trabalho 1. A medida que decide a questão é a **contagem de comparações** da seção 3.2.3, que cresce exatamente 4,00× a cada duplicação, sem exceção.

3.2.2 Tempo de pesquisa e de remoção no pior caso

n	Pesquisa por telefone (ms) árvore balanceada	razão	Pesquisa por telefone (ms) árvore degenerada	razão
25.000	0,0012	—	0,3333	—
50.000	0,0013	1,08×	0,8680	2,60×
100.000	0,0017	1,31×	1,4427	1,66×
200.000	0,0019	1,12×	2,8511	1,98×

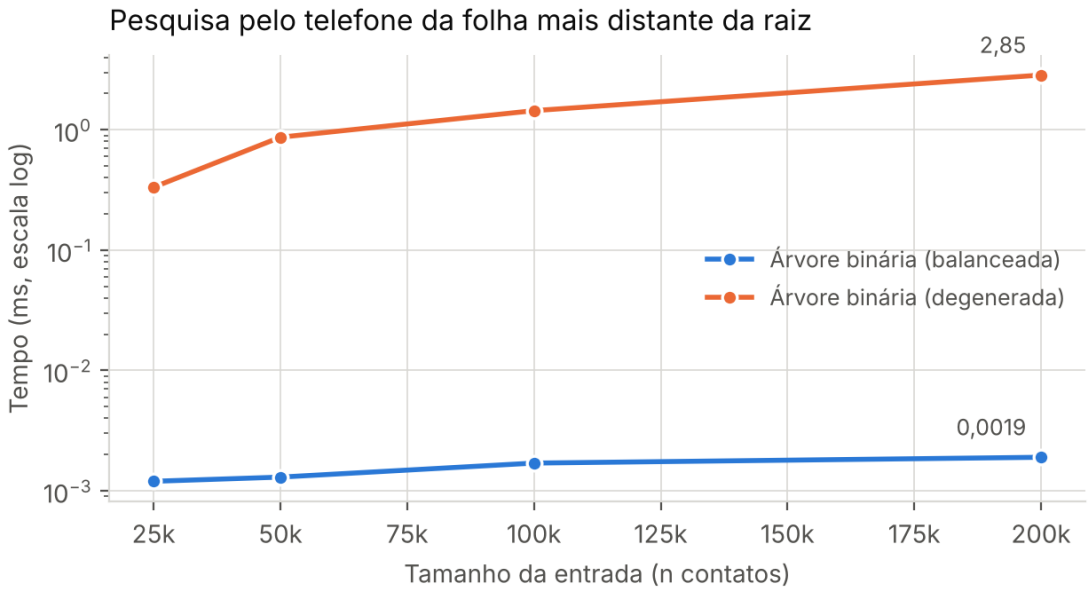


Figura 3 — Pesquisa pelo telefone da folha mais distante da raiz.

n	Pesquisa por nome (ms) árvore balanceada	razão	Pesquisa por nome (ms) árvore degenerada	razão
25.000	0,0012	—	0,3204	—
50.000	0,0013	1,08×	1,6930	5,28×
100.000	0,0017	1,31×	1,7273	1,02×
200.000	0,0020	1,18×	3,1234	1,81×

n	Remoção (ms) árvore balanceada	razão	Remoção (ms) árvore degenerada	razão
25.000	0,0014	—	1,5334	—
50.000	0,0014	1,00×	0,9496	0,62×
100.000	0,0019	1,36×	1,4456	1,52×
200.000	0,0022	1,16×	2,8486	1,97×

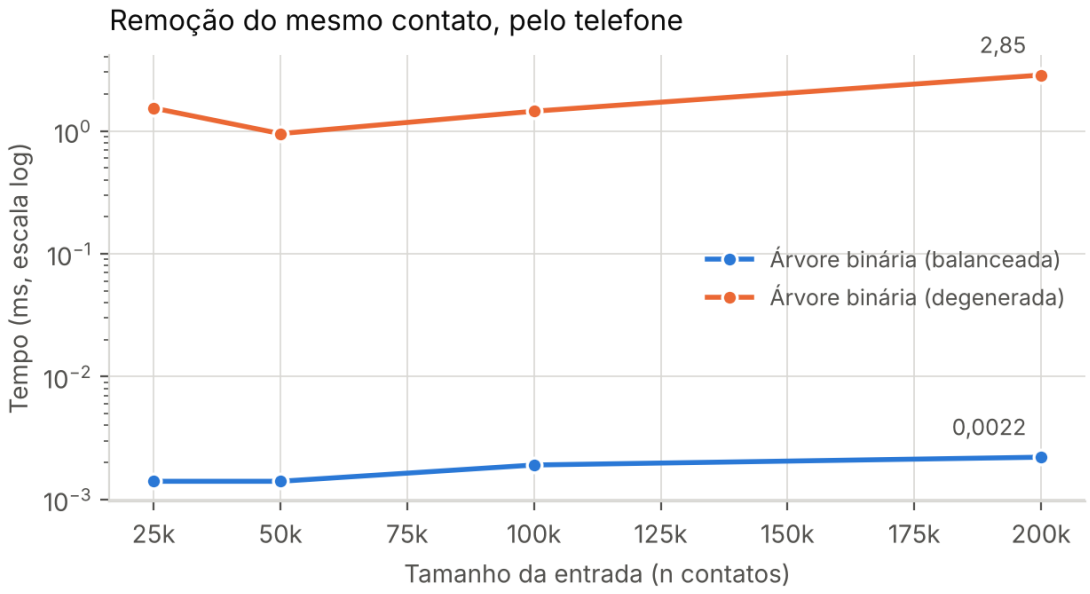


Figura 4 — Remoção do mesmo contato, pesquisado pelo telefone.

Interpretação. As três tabelas contam a mesma história, e é a prevista na seção 2. Na árvore **balanceada**, as duas buscas e a remoção custam cerca de **um microssegundo** (0,001 ms) e praticamente não crescem: entre 25.000 e 200.000 contatos — oito vezes mais dados — a altura sobe apenas de 14 para 17, e é a altura que determina o custo. É o comportamento **O(log n)**: multiplicar *n* por 8 acrescenta 3 níveis à árvore, e portanto 3 comparações à busca.

Na árvore **degenerada**, as mesmas operações custam **milissegundos** — duas a três ordens de grandeza a mais — e crescem com *n*, o comportamento **O(n)**, porque percorrer a árvore degenerada é percorrer uma lista. A confirmação exata vem da contagem de operações (seção 3.2.3): a busca de pior caso faz **exatamente n comparações** na árvore degenerada e **h + 1** na balanceada.

Por que as razões de tempo oscilam. Em termos absolutos, essas operações duram de microssegundos (balanceada) a poucos milissegundos (degenerada) — faixa em que o relógio sofre interferência do escalonador do sistema operacional, do coletor de lixo e do estado dos caches. Mesmo com a mediana de 15 medições por ponto, algumas razões ficam abaixo de 1, ou seja, o tempo medido **caiu** ao dobrar *n*. Isso não contradiz a análise: significa apenas que, nessa escala, a variação do ruído é maior que a diferença real entre dois pontos vizinhos. A tendência entre os extremos (25.000 e 200.000) e a contagem de comparações são as evidências confiáveis, e ambas confirmam O(log n) e O(n).

Dois pontos específicos que merecem comentário. Primeiro, a **remoção na árvore degenerada** é consistentemente próxima do tempo da busca pelo mesmo elemento. Isso é esperado: a remoção é uma busca seguida de um número fixo de religações de ponteiros, e nesta topologia o nó removido é o último da espinha — uma folha, portanto o caso mais barato da segunda fase, sem predecessor a procurar. Segundo, nos tamanhos menores a **busca por nome** chega a ser mais rápida que a busca por telefone na degenerada, embora as duas árvores tenham a mesma topologia e a mesma altura. O motivo é que a busca por nome roda logo depois da busca por telefone dentro de cada repetição, aproveitando o aquecimento do JIT; nos tamanhos maiores, em que o trabalho real domina o ruído, as duas praticamente se igualam, como deveria ser. É um ganho de fator constante, não de ordem.

3.2.3 Contagem de operações

A tabela mostra quantas **comparações entre contatos** foram efetivamente executadas para montar as duas árvores (soma das duas) e para fazer a busca de pior caso.

n	Montagem — ABB balanceada (soma das 2 árvores)	razão	Montagem — ABB degenerada (soma das 2 árvores)	razão	Busca — ABB balanceada (1 árvore)	Busca — ABB degenerada (1 árvore)
25.000	634.496	—	624.975.000	—	15	25.000
50.000	1.368.962	2,16×	2.499.950.000	4,00×	16	50.000
100.000	2.937.892	2,15×	9.999.900.000	4,00×	17	100.000
200.000	6.275.750	2,14×	39.999.800.000	4,00×	18	200.000

Os números confirmam a análise matemática com precisão. Na árvore **degenerada** a contagem é multiplicada por **4,00** a cada duplicação de *n* — a assinatura de n^2 —, e mais do que isso: ela bate **exatamente** com a fórmula deduzida na seção 2.2. Lá chegamos a n(n–1)/2 comparações por árvore; como montamos duas árvores, o total previsto é n(n–1). Para n = 25.000 isso dá 624.975.000, e foi exatamente esse o valor medido — até a última unidade. O mesmo vale para os demais tamanhos.

Na árvore **balanceada** a contagem cresce por pouco mais de **2** a cada duplicação: é o comportamento n·log n, em que dobrar *n* dobra o número de inserções e ainda acrescenta um nível a cada uma. E a busca de pior caso confirma a previsão da seção 2.3 nos dois extremos: na balanceada ela faz **h + 1** comparações (15, 16, 17 e 18 para alturas 14, 15, 16 e 17), e na degenerada faz **exatamente n** comparações — o pior caso linear acontecendo literalmente.

3.2.4 Comparação com as listas do Trabalho 1

Para que a comparação fosse justa, rodamos também as listas encadeadas do Trabalho 1 **na mesma máquina e com os mesmos arquivos**. Tempos de montagem, em milissegundos:

n	Árvore balanceada	Árvore degenerada	Lista não-ordenada	Lista ordenada
25.000	46,0	4.888,7	36,3	3.159,8
50.000	62,0	27.967,1	44,2	13.196,7
100.000	101,4	132.331,6	91,4	61.210,6
200.000	168,5	464.520,5	121,1	363.437,9

Tempos de pesquisa no pior caso, em milissegundos:

n	Árvore balanceada	Árvore degenerada	Lista não-ordenada	Lista ordenada
25.000	0,0012	0,3333	0,5773	0,6398
50.000	0,0013	0,8680	0,9942	1,0733
100.000	0,0017	1,4427	1,6090	1,5625
200.000	0,0019	2,8511	2,1618	8,6214

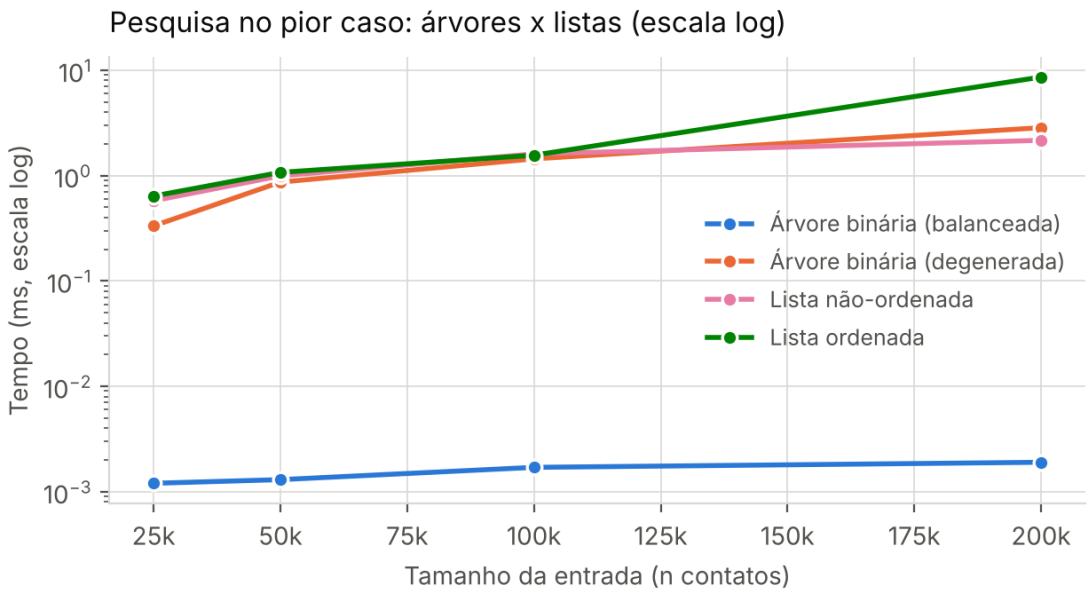


Figura 5 — Pesquisa no pior caso: as quatro estruturas (escala logarítmica).

O que a comparação mostra. Para a **busca**, a árvore balanceada é disparadamente a melhor estrutura: ela resolve em microssegundos o que as listas resolvem em milissegundos, e a vantagem **aumenta** com *n*, porque O(log n) e O(n) divergem. A árvore degenerada, por sua vez, tem desempenho equivalente ao da lista — o que era de se esperar, já que ela **é** uma lista encadeada com um ponteiro desperdiçado por nó.

Para a **montagem**, a lista não-ordenada continua imbatível (O(n), com inserção O(1) no fim graças à referência `ult`), mas ela paga esse preço em toda busca posterior. A árvore balanceada monta em O(n log n) — mais caro que a lista não-ordenada e muito mais barato que a lista ordenada ou que a árvore degenerada, ambas quadráticas.

Duas observações sobre a leitura dessas tabelas. A primeira é que as medidas da **lista não-ordenada** na busca também oscilam entre tamanhos vizinhos, pelo mesmo motivo já discutido na seção 3.2.2 — são poucos milissegundos, e o ruído de medição é da mesma ordem da diferença entre dois pontos. A tendência entre os extremos, no entanto, é claramente linear, como manda o O(n) da busca pelo último nó. A segunda é que a **lista ordenada** foi medida com o arquivo balanceado, e não com o crescente: montar uma lista ordenada a partir de chaves em ordem crescente é o pior caso absoluto dessa estrutura (cada inserção percorre a lista inteira), e o tempo seria ainda maior que o já medido — o que não acrescentaria nada à comparação, já que o ponto é o mesmo da árvore degenerada.

A conclusão prática é que **a escolha da estrutura depende do perfil de uso**: se os dados são inseridos uma vez e consultados muitas, a árvore balanceada compensa largamente o custo de montagem. Mas essa conclusão tem uma condição embutida — que a árvore **esteja** balanceada —, e nada no código garante isso: basta o arquivo chegar ordenado para cairmos no pior caso.

3.3 Conclusão da análise da árvore binária

Os dados coletados **atendem ao esperado** pela análise matemática da seção 2:

• as alturas medidas bateram exatamente com os limites teóricos — piso($\log_2$ n) na árvore gerada pelo arquivo balanceado e n–1 na gerada pelo arquivo crescente;
• a **montagem** da árvore balanceada cresceu como O(n log n) e a da degenerada como O(n^2), com

a contagem de comparações quadruplicando a cada duplicação de *n*;
• **pesquisa** e **remoção** no pior caso ficaram em microssegundos na árvore balanceada e em milissegundos na degenerada — diferença de duas a três ordens de grandeza —, e a contagem de comparações confirmou O(log n) e O(n) com exatidão (h+1 e n comparações, respectivamente), ainda que os tempos de relógio oscilem entre tamanhos vizinhos por serem medidas muito curtas;
• `quantidadeNos` é O(1) nas duas topologias e seu tempo é indistinguível de zero, por isso não foi incluído nos gráficos.

A lição central desta etapa é que, em uma árvore binária de busca **sem balanceamento**, a complexidade não é uma propriedade do algoritmo isoladamente: ela depende da **ordem em que os dados chegam**. O mesmo código, com os mesmos 200.000 contatos, foi de 17 níveis para 199.999 e de milissegundos para minutos. Em um sistema real, dados costumam chegar ordenados (por data de cadastro, por matrícula, por código sequencial), que é justamente o pior caso.

É exatamente esse problema que as **árvores AVL** resolvem, reequilibrando a árvore após cada inserção e remoção para garantir h = O(log n) em qualquer ordem de entrada. É o que analisamos na seção 4 e medimos na seção 5.

4. Árvore AVL: implementação e análise de complexidade

A seção anterior terminou com um problema concreto: a árvore binária de busca só entrega O(log n) **se** a ordem de chegada dos dados colaborar, e a ordem mais comum em sistemas reais — chaves sequenciais — é justamente a pior possível. A árvore **AVL** (de Adelson-Velsky e Landis) resolve isso verificando o equilíbrio após cada inserção e cada remoção e corrigindo-o imediatamente, de modo que a altura permaneça O(log n) **para qualquer ordem de entrada**.

4.1 O critério de equilíbrio

Para cada nó define-se o **fator de balanceamento**:

```
      fb(no) = altura(subarvore esquerda) - altura(subarvore direita)

  Uma arvore e AVL quando |fb| <= 1 em TODOS os nos, ou seja, as duas
  subarvores de qualquer no diferem em no maximo um nivel de altura.

  Consequencia (limite classico da AVL):

      h  <  1,4404 * log2(n + 2) - 0,3277

  isto e, a altura de uma AVL nunca passa de cerca de 44% A MAIS que a
  altura minima teorica piso(log2 n). Ela nao e perfeitamente balanceada, mas e
  garantidamente O(log n) - que e o que importa para a complexidade.
```

Na nossa implementação, cada nó guarda a sua própria altura (campo `altura` de `NoArvore`, folha = 0) e o fator de balanceamento é derivado dela. É a mesma informação do campo `fb` dos exemplos em C vistos em aula, apenas armazenada de outra forma: guardar a altura torna a atualização trivial tanto na inserção quanto na remoção, enquanto manter o `fb` diretamente exige propagar sinalizadores de “cresceu/diminuiu” subindo pela recursão.

Listagem 7 — altura do nó e fator de balanceamento — src/arvorebinaria/ArvoreAVL.java, linhas 127-142

```
127     /** Altura da subárvore; -1 para subárvore vazia. */
128     private int alturaDe(NoArvore<T> no) {
129         return (no == null) ? -1 : no.getAltura();
130     }
131
132     private void atualizarAltura(NoArvore<T> no) {
133         no.setAltura(1 + Math.max(alturaDe(no.getEsq()), alturaDe(no.getDir())));
134     }
135
136     /**
137      * @return o fator de balanceamento do nó: altura da subárvore esquerda
138      *         menos altura da subárvore direita.
139      */
140     private int fatorBalanceamento(NoArvore<T> no) {
141         return alturaDe(no.getEsq()) - alturaDe(no.getDir());
142     }
```

4.2 As quatro rotações

Quando um nó fica com |fb| = 2, o equilíbrio é restaurado por uma ou duas **rotações** — operações que reorganizam três nós e no máximo quatro subárvores, preservando a propriedade de árvore de busca. Cada rotação mexe em um número **fixo** de ponteiros: é O(1).

Os quatro casos

```
CASO ESQUERDA-ESQUERDA (fb do no = +2, fb do filho esquerdo >= 0)
   rotacao simples a DIREITA

        30                       20
       /                        /  \
      20          ==>          10    30
     /
    10

CASO DIREITA-DIREITA (fb do no = -2, fb do filho direito <= 0)
   rotacao simples a ESQUERDA  -- e o caso da nossa entrada crescente

    10                           20
      \                         /  \
       20        ==>           10    30
         \
          30

CASO ESQUERDA-DIREITA (fb do no = +2, fb do filho esquerdo < 0)
   rotacao dupla: primeiro a ESQUERDA no filho, depois a DIREITA no no

        30            30             20
       /             /              /  \
      10    ==>     20     ==>     10    30
        \          /
         20       10

CASO DIREITA-ESQUERDA (fb do no = -2, fb do filho direito > 0)
   rotacao dupla: primeiro a DIREITA no filho, depois a ESQUERDA no no

    10              10                  20
      \               \                /  \
       30    ==>       20     ==>     10    30
      /                  \
     20                   30
```

Listagem 8 — escolha da rotação — src/arvorebinaria/ArvoreAVL.java, linhas 144-165

```
144     /**
145      * Aplica, se necessário, uma das quatro rotações que restauram a
146      * propriedade AVL no nó recebido.
147      */
148     private NoArvore<T> balancear(NoArvore<T> no) {
149         int fb = fatorBalanceamento(no);
150         if (fb > 1) {
151             // Desequilíbrio à esquerda.
152             if (fatorBalanceamento(no.getEsq()) < 0) {
153                 no.setEsq(rotacaoEsquerda(no.getEsq()));   // caso esquerda-direita
154             }
155             return rotacaoDireita(no);                     // caso esquerda-esquerda
156         }
157         if (fb < -1) {
158             // Desequilíbrio à direita.
159             if (fatorBalanceamento(no.getDir()) > 0) {
160                 no.setDir(rotacaoDireita(no.getDir()));    // caso direita-esquerda
161             }
162             return rotacaoEsquerda(no);                    // caso direita-direita
163         }
164         return no;
165     }
```

O método `balancear` é chamado em **todo** nó do caminho de volta da recursão, tanto na inserção quanto na remoção. Ele olha o fator de balanceamento e, se |fb| ≤ 1, devolve o nó sem tocar em nada (linha 164); caso contrário aplica a rotação simples ou a dupla, conforme o sinal do fb do filho.

Listagem 9 — rotação simples à direita — src/arvorebinaria/ArvoreAVL.java, linhas 177-184

```
177      private NoArvore<T> rotacaoDireita(NoArvore<T> no) {
178          NoArvore<T> esq = no.getEsq();
179          no.setEsq(esq.getDir());
180          esq.setDir(no);
181          atualizarAltura(no);
182          atualizarAltura(esq);
183          return esq;
184      }
```

A rotação é apenas um religamento de ponteiros seguido da atualização das duas alturas afetadas: cinco operações, nenhuma delas dependente de *n*. **Toda rotação é O(1)** — esse é o ponto que sustenta a análise a seguir.

4.3 Método adicionar

Listagem 10 — adicionar na AVL — src/arvorebinaria/ArvoreAVL.java, linhas 43-64

```
43      @Override
44      public boolean adicionar(T novoValor) {
45          if (novoValor == null) {
46              return false;
47          }
48          this.raiz = inserir(this.raiz, novoValor);
49          this.quant++;
50          return true;
51      }
52
53      private NoArvore<T> inserir(NoArvore<T> no, T novoValor) {
54          if (no == null) {
55              return new NoArvore<>(novoValor);
56          }
57          if (comparador.compare(no.getValor(), novoValor) > 0) {
58              no.setEsq(inserir(no.getEsq(), novoValor));
59          } else {
60              no.setDir(inserir(no.getDir(), novoValor));
61          }
62          atualizarAltura(no);
63          return balancear(no);
64      }
```

A inserção tem duas fases. A **descida** (linhas 57-61) é idêntica à da árvore binária comum: compara e desce para um dos lados até encontrar o lugar do novo nó. A **volta da recursão** (linhas 62-63) é a novidade: ao retornar de cada nível, a altura do nó é atualizada e o nó é rebalanceado.

Custo. A descida percorre *h* níveis. Na volta, cada um dos *h* níveis executa `atualizarAltura` e `balancear`, ambos O(1). Logo **T(n) = c·h**, ou seja, **O(h)**. Como a AVL garante h = O(log n), a inserção é **O(log n) no pior caso** — e, diferentemente da árvore binária comum, **não existe ordem de entrada capaz de piorar isso**.

Quantas rotações a inserção faz? No máximo **uma** rotação simples ou **uma** dupla, e depois dela nenhum ancestral precisa de correção. O motivo é que a rotação devolve à subárvore a mesma altura que ela tinha antes da inserção, de modo que os nós acima deixam de enxergar qualquer mudança. Por isso o rebalanceamento acrescenta apenas uma constante ao custo: O(log n) para descer, O(1) para corrigir.

Comparando com a árvore binária comum: no melhor caso (entrada balanceada) as duas são O(log n) e a AVL é um pouco mais lenta, porque paga a atualização de altura em cada nível. No **pior caso**, porém, a árvore comum vai a O(n) e a AVL permanece em O(log n). Montar *n* elementos custa **O(n log n)** na AVL contra **O(n^2)** na árvore comum degenerada.

4.4 Método remover

Listagem 11 — remover na AVL — src/arvorebinaria/ArvoreAVL.java, linhas 70-111

```
 70      @Override
 71      public boolean remover(T valor) {
 72          this.removeuElemento = false;
 73          this.raiz = remover(this.raiz, valor);
 74          if (this.removeuElemento) {
 75              this.quant--;
 76          }
 77          return this.removeuElemento;
 78      }
 79
 80      private NoArvore<T> remover(NoArvore<T> no, T valor) {
 81          if (no == null) {
 82              return null;
 83          }
 84          int cmp = comparador.compare(no.getValor(), valor);
 85          if (cmp > 0) {
 86              no.setEsq(remover(no.getEsq(), valor));
 87          } else if (cmp < 0) {
 88              no.setDir(remover(no.getDir(), valor));
 89          } else {
 90              this.removeuElemento = true;
 91              // Caso 1 e 2: nó folha ou com um único filho.
 92              if (no.getEsq() == null) {
 93                  return no.getDir();
 94              }
 95              if (no.getDir() == null) {
 96                  return no.getEsq();
 97              }
 98              // Caso 3: dois filhos. Mesma estratégia da árvore binária comum —
 99              // o nó recebe o valor do predecessor (o maior da subárvore
100              // esquerda) e o predecessor é removido da subárvore esquerda.
101              NoArvore<T> predecessor = no.getEsq();
102              while (predecessor.getDir() != null) {
103                  predecessor = predecessor.getDir();
104              }
105              T valorPredecessor = predecessor.getValor();
106              no.setValor(valorPredecessor);
107              no.setEsq(removerPredecessor(no.getEsq()));
108          }
109          atualizarAltura(no);
110          return balancear(no);
111      }
```

A remoção segue a mesma estrutura da árvore binária comum — busca o nó (linhas 84-88) e trata os três casos: folha ou nó com um único filho (linhas 91-96) e nó com dois filhos, resolvido pelo **predecessor**, o maior elemento da subárvore esquerda (linhas 98-107) — e acrescenta, na volta da recursão, a mesma dupla `atualizarAltura` + `balancear` (linhas 109-110).

Note o método auxiliar `removerPredecessor`: ele desce pela direita até o último nó e também rebalanceia na volta. Isso é necessário porque a remoção do predecessor pode desequilibrar os nós daquele caminho.

Custo. Busca O(h), remoção propriamente dita O(1), rebalanceamento O(1) por nível em um caminho de *h* níveis: **O(h) = O(log n)** no pior caso.

Uma diferença importante em relação à inserção: enquanto a inserção precisa de no máximo uma rotação, a remoção pode precisar de **até h rotações** — uma em cada nível do caminho de volta. Isso acontece porque uma rotação que reequilibra uma subárvore pode **reduzir** a altura dela, e essa redução propaga o desequilíbrio para o pai, e assim por diante até a raiz. Mesmo no pior caso, porém, são O(log n) rotações de O(1) cada, e o custo total da remoção continua **O(log n)**.

Esse detalhe costuma ser cobrado em prova: **inserção = no máximo 1 rotação; remoção = até O(log n) rotações**; as duas operações, ainda assim, O(log n).

4.5 Resumo da seção

Método	Árvore binária (pior caso)	AVL (pior caso)	Comentário
adicionar	O(n)	**O(log n)**	A AVL paga O(1) de rebalanceamento por nível e, em troca, elimina o pior caso.
pesquisar	O(n)	**O(log n)**	Herdado sem alteração: a AVL continua sendo uma árvore de busca; o que muda é a altura garantida.
remover	O(n)	**O(log n)**	Até O(log n) rotações, cada uma O(1).
quantidadeNos	O(1)	O(1)	Contador mantido nas duas classes.
montar n elementos	$O(n^2)$	**O(n log n)**	Independentemente da ordem de chegada dos dados.

O preço do equilíbrio é um campo a mais por nó (a altura) e algumas operações constantes por nível. O benefício é eliminar completamente a dependência da ordem de entrada — que, como a seção 3 mostrou, pode custar minutos em vez de milissegundos.

5. Desempenho da AVL comparado ao da árvore binária comum

Esta seção repete exatamente as medições da seção 3, porém instanciando **árvores AVL** e usando **os mesmos arquivos em ordem crescente** — aqueles que, na árvore binária comum, produzem a árvore completamente degenerada. A metodologia, a máquina e o critério de pior caso são os mesmos descritos na seção 3.1: a busca é feita pela folha mais distante da raiz na árvore indexada por telefone, e o mesmo contato é usado nas três operações.

5.1 Altura: o efeito do balanceamento

A medida que explica todo o resto. Com a **mesma entrada**:

n	Altura — AVL	Altura — árvore binária	piso($\log_2$ n)	n − 1
25.000	14	24.999	14	24.999
50.000	15	49.999	15	49.999
100.000	16	99.999	16	99.999
200.000	17	199.999	17	199.999

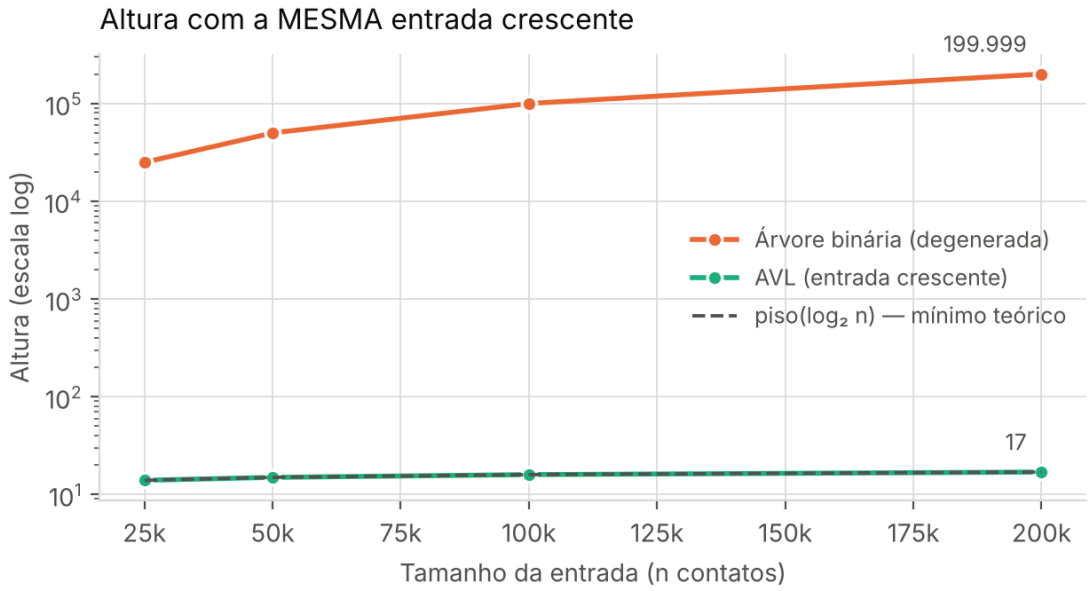


Figura 6 — Altura com a mesma entrada crescente (escala logarítmica). A AVL coincide com o mínimo teórico; a árvore binária comum vai a n−1.

O rebalanceamento funcionou exatamente como a teoria prevê. Com 200.000 contatos em ordem crescente, a árvore binária comum chega a **199.999** níveis e a AVL fica em **17** — que é o próprio piso($\log_2$ n), o mínimo teórico possível para essa quantidade de nós. Vale notar que a AVL não *garante* atingir o mínimo (o limite da seção 4.1 permite até cerca de 44% a mais), mas para uma entrada crescente ela acaba produzindo exatamente a árvore mais rasa possível.

5.2 Tempo de montagem

n	AVL (entrada crescente)	razão	Árvore binária (entrada crescente)	razão	ganho
25.000	50,56	—	4.888,70	—	97×
50.000	68,50	1,35×	27.967,14	5,72×	408×
100.000	106,61	1,56×	132.331,62	4,73×	1.241×
200.000	186,70	1,75×	464.520,49	3,51×	2.488×

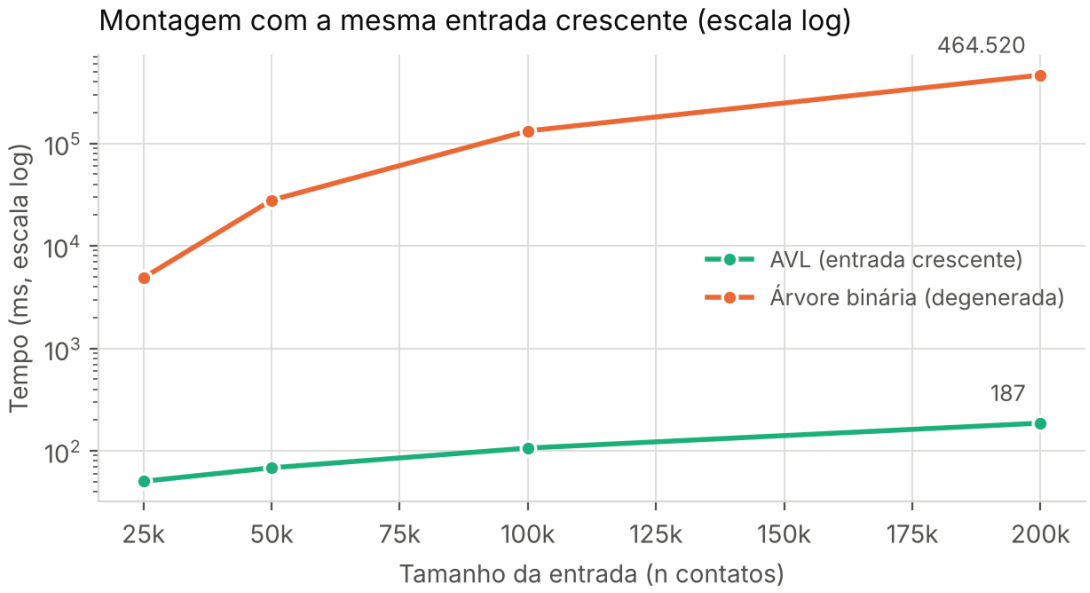


Figura 7 — Montagem com a mesma entrada crescente (escala logarítmica).

As razões da AVL são 1,35×, 1,56× e 1,75× a cada duplicação de *n* — próximas do fator teórico de O(n log n), que para esses tamanhos é pouco acima de 2, e abaixo dele pelos mesmos custos fixos discutidos na seção 3.2.1 (compilação JIT e leitura do arquivo pesam proporcionalmente mais nas entradas menores). Já a árvore binária comum, com a **mesma** entrada, apresenta razões 5,72×, 4,73× e 3,51× — oscilando em torno de 4, o **O(n^2)** da seção 3.

Em números absolutos: com 200.000 contatos em ordem crescente, a AVL montou as duas árvores em **187 ms**, contra **7,7 minutos** da árvore binária comum: um ganho de cerca de **2.488 vezes**. E, como a diferença entre O(n log n) e O(n^2) cresce com *n*, esse fator **aumenta** conforme a base de dados cresce.

5.3 Contagem de operações

Como nas seções anteriores, a contagem de comparações é a evidência livre de ruído. Aqui ela permite comparar, com a **mesma entrada crescente**, quantas comparações cada estrutura executa:

n	Comparações — AVL (soma das 2 árvores)	razão	Comparações — ABB (soma das 2 árvores)	razão	Busca — AVL (1 árvore)
25.000	684.466	—	624.975.000	—	15
50.000	1.468.930	2,15×	2.499.950.000	4,00×	16
100.000	3.137.858	2,14×	9.999.900.000	4,00×	17
200.000	6.675.714	2,13×	39.999.800.000	4,00×	18

A leitura é direta: com 200.000 contatos em ordem crescente, a AVL executa cerca de **6,7 milhões** de comparações para montar as duas árvores, contra **40 bilhões** da árvore binária comum — uma diferença de quase quatro ordens de grandeza, e que cresce com *n*. As razões confirmam as ordens previstas: a AVL pouco mais que dobra a cada duplicação de *n* (n·log n) e a árvore comum quadruplica (n^2).

Vale notar também a comparação entre a AVL montada a partir da entrada **crescente** e a árvore binária **balanceada** da seção 3: para 200.000 elementos, 6.675.714 comparações contra 6.275.750, ou seja, a AVL faz de **6% a 8% mais comparações** (7,9% em 25.000 e 6,4% em 200.000). A diferença não é o custo das rotações em si — uma rotação religa ponteiros e **não chama o comparador** — mas o fato de a árvore produzida pelas rotações não ser exatamente a mesma árvore perfeitamente balanceada: alguns elementos acabam um nível mais abaixo, e cada nível extra é uma comparação a mais na inserção dos elementos seguintes.

A contraprova está em contar as comparações da AVL com o arquivo **já balanceado**: 634.496, 1.368.962, 2.937.892 e 6.275.750 — valores **idênticos**, dígito a dígito, aos da árvore binária comum com o mesmo arquivo. Quando a entrada já chega equilibrada, nenhuma rotação é disparada e a AVL executa exatamente a mesma sequência de comparações. Isso isola o custo do balanceamento: em número de comparações ele é **zero**; em tempo, como se verá na seção 5.5, ele é de 12% a 16%, e vem inteiramente das atualizações de altura e das verificações de fator de balanceamento, que não são comparações.

5.4 Tempo de pesquisa e de remoção no pior caso

n	AVL — pesquisa por telefone (ms)	AVL — pesquisa por nome (ms)	AVL — remoção (ms)	Árvore binária — pesquisa (ms)	Árvore binária — remoção (ms)
25.000	0,0012	0,0012	0,0022	0,3333	1,5334
50.000	0,0013	0,0013	0,0023	0,8680	0,9496
100.000	0,0014	0,0014	0,0023	1,4427	1,4456
200.000	0,0015	0,0015	0,0027	2,8511	2,8486

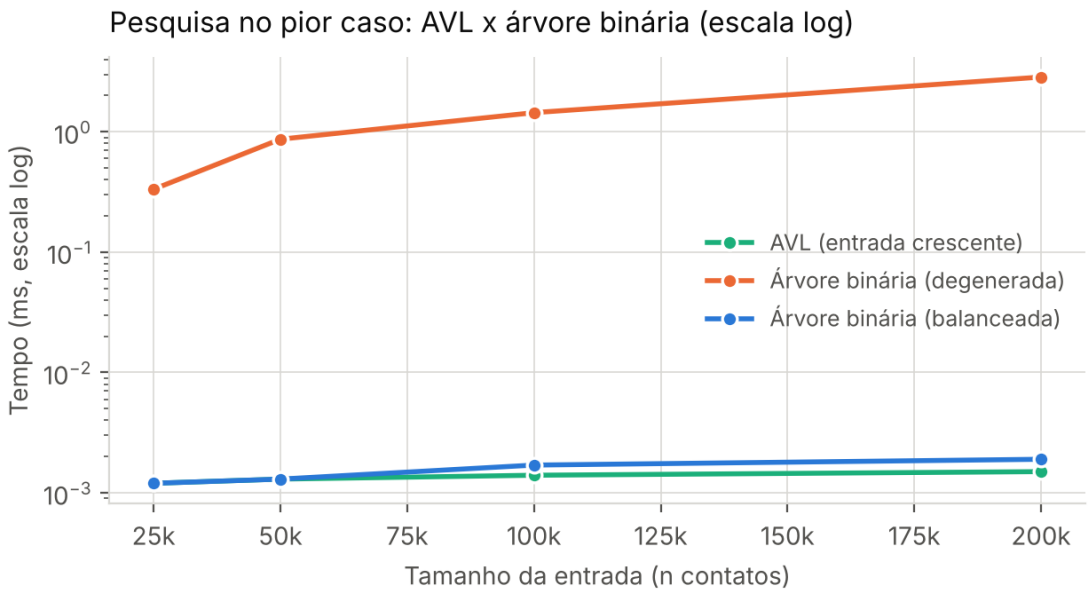


Figura 8 — Pesquisa no pior caso (escala logarítmica). A AVL construída a partir da entrada crescente acompanha a árvore binária balanceada.

Na AVL, as três operações — busca pelo telefone da folha mais distante da raiz, busca do **mesmo contato pelo nome** na segunda árvore e remoção desse contato — custam **cerca de um microssegundo** e praticamente não crescem com *n*: o mesmo patamar da árvore binária **balanceada** medida na seção 3, e o esperado para O(log n). A busca por nome é a mais barata das três (de 0,0012 a 0,0015 ms) porque percorre a árvore indexada por nome, que com esta entrada tem a mesma altura da outra, sem nenhum religamento de ponteiros a fazer. Na árvore binária comum com a mesma entrada, as mesmas operações custam **milissegundos** e crescem proporcionalmente a *n*.

A conclusão da comparação é que a AVL entrega, para a entrada *pior possível* da árvore binária comum, exatamente o desempenho que a árvore comum só alcança na entrada *melhor possível*. Em outras palavras: a AVL troca uma dependência perigosa — a sorte na ordem de chegada dos dados — por um custo fixo e pequeno de manutenção.

5.5 Os dados atendem ao previsto na seção 4?

Confrontando cada previsão da análise matemática com o que foi medido:

Previsão da seção 4	O que foi medido	Confere?
A AVL mantém h = O(log n) para qualquer ordem de entrada (4.1)	Com a entrada crescente — a pior para a árvore comum — a AVL ficou em 14, 15, 16 e 17 níveis, exatamente piso($\log_2$ n).	**Sim**
adicionar é O(log n), logo montar n elementos é O(n log n) (4.3)	As razões de tempo da montagem ficaram entre 1,35× e 1,75× por duplicação de n, e a contagem de comparações cresceu cerca de 2,14× — compatível com n·log n, e muito distante do 4,00× quadrático da árvore comum.	**Sim**
pesquisar é O(log n), fazendo h + 1 comparações no pior caso (4.3 e 2.3)	A contagem deu exatamente 15, 16, 17 e 18 comparações para alturas 14, 15, 16 e 17, e o tempo ficou em torno de 1,5 microssegundo, praticamente constante.	**Sim**

Previsão da seção 4	O que foi medido	Confere?
remover é O(log n), ainda que possa exigir até O(log n) rotações (4.4)	A remoção custou de 2,2 a 2,7 microssegundos, cerca do dobro da busca — coerente com busca mais rebalanceamento na volta — e praticamente não cresceu com n.	**Sim**
O preço do equilíbrio é um custo constante por nível (4.5)	Com o arquivo já balanceado, em que nenhuma rotação é disparada, a AVL fez **exatamente o mesmo número de comparações** da árvore comum (634.496 a 6.275.750) e custou de 12% a 16% a mais de **tempo** — ou seja, o sobrecusto é só o das atualizações de altura, constante por nível.	**Sim**

Nenhuma previsão da seção 4 foi contrariada pelos dados. A única observação que vale registrar é sobre a **precisão das medidas de tempo** das operações individuais: elas duram poucos microssegundos na AVL, faixa em que o relógio sofre interferência do sistema operacional e do coletor de lixo. Por isso a contagem de comparações da seção 5.3 é a evidência mais forte do O(log n) — e ela é exata.

E qual é esse custo? Ele aparece quando comparamos a AVL com a árvore binária na entrada **balanceada**, em que o rebalanceamento não é necessário e portanto só acrescenta trabalho:

n	Montagem — árvore binária (ms)	Montagem — AVL (ms)	Sobrecusto
25.000	46,04	51,35	+12%
50.000	61,95	69,45	+12%
100.000	101,38	118,05	+16%
200.000	168,45	192,15	+14%

Com a entrada já balanceada, as duas estruturas chegam à mesma altura e a AVL faz trabalho que não era necessário: atualizar a altura de cada nó no caminho de volta e verificar o fator de balanceamento em cada um. O sobrecusto medido fica em **12% a 16% no tempo de montagem**. Esse sobrecusto é **inteiramente** composto por essas operações de altura: como mostrado na seção 5.3, a contagem de comparações da AVL com esse mesmo arquivo é idêntica à da árvore comum, porque nenhuma rotação chega a ser disparada. É um preço pago uma única vez, na montagem, e um seguro barato contra um pior caso que custa minutos.

6. Árvores binárias na biblioteca padrão de Java

A biblioteca padrão de Java não expõe uma classe “árvore binária de busca” de uso geral, nem uma AVL. O que ela oferece são **coleções ordenadas implementadas sobre uma árvore binária balanceada** — e a escolha de qual árvore usar é, por si só, uma resposta interessante à pergunta desta seção.

6.1 TreeMap e TreeSet: árvores rubro-negras

A classe `java.util.TreeMap` é descrita na própria documentação oficial como “*A Red-Black tree based NavigableMap implementation*”, e acrescenta que os algoritmos são adaptados do livro *Introduction to Algorithms*, de Cormen, Leiserson e Rivest. Já `java.util.TreeSet` não implementa árvore nenhuma: é “*a NavigableSet implementation based on a TreeMap*”, ou seja, um conjunto construído sobre um `TreeMap` em que só as chaves importam.

Garantias de desempenho. A documentação do `TreeMap` afirma: “*This implementation provides guaranteed log(n) time cost for the containsKey, get, put and remove operations*”. A do `TreeSet` declara o mesmo para `add`, `remove` e `contains`. Note a palavra **guaranteed**: é a mesma promessa da nossa AVL e justamente aquela que a árvore binária comum **não** pode fazer.

6.2 O que é uma árvore rubro-negra

É uma árvore binária de busca em que cada nó carrega uma cor (vermelho ou preto) e em que valem as seguintes regras:

```
1. Todo no e vermelho ou preto.
2. A raiz e preta.
3. Toda folha (no nulo) e considerada preta.
4. Um no vermelho nao pode ter filho vermelho (nao ha dois vermelhos
   seguidos em um caminho).
5. Todo caminho de um no ate qualquer folha descendente contem o MESMO
   numero de nos pretos.

Consequencia: o caminho mais longo da raiz ate uma folha tem no maximo o
DOBRO do comprimento do mais curto, o que limita a altura a

                    h <= 2 * log2(n + 1)

ou seja, O(log n) - como na AVL, porem com uma folga maior.
```

O equilíbrio é mantido por **recolorações** e **rotações** — as mesmas rotações simples e duplas que implementamos na AVL (seção 4.2). A diferença é quando elas são acionadas: a rubro-negra tolera desequilíbrios maiores e, por isso, precisa reorganizar a estrutura com menos frequência.

6.3 Como os elementos são comparados

Exatamente como na nossa biblioteca. A documentação do `TreeMap` diz que “*The map is sorted according to the natural ordering of its keys, or by a Comparator provided at map creation time, depending on which constructor is used*”. São os dois caminhos vistos em aula:

Construtor	Critério de comparação	Exige do tipo armazenado
`new TreeMap<>()`	Ordem natural da chave	Que a chave implemente `Comparable` (senão, `ClassCastException` na inserção)
`new TreeMap<>(cmp)`	O `Comparator` informado	Nada — é o comparador que sabe comparar, como na nossa `ArvoreBinaria`

Há ainda uma exigência sutil que vale conhecer: a documentação alerta que a ordem "*must be consistent with equals*", porque "*a sorted map performs all key comparisons using its compareTo (or compare) method, so two keys that are deemed equal by this method are, from the standpoint of the sorted map, equal*". Em outras palavras: se o comparador disser que dois objetos diferentes são iguais, o `TreeMap` os tratará como a mesma chave. É o mesmo cuidado que tivemos ao escolher o telefone — um valor único — como chave de uma das nossas árvores.

6.4 Por que o Java escolheu rubro-negra em vez de AVL

As duas garantem O(log n), então a escolha é uma troca entre dois custos:

Critério	AVL	Rubro-negra
Critério de equilíbrio	$\|fb\| \leq 1$ em todo nó	Regras de cor; caminho mais longo ≤ 2× o mais curto
Altura máxima	$\approx 1{,}44 \cdot \log_2 n$	$\approx 2 \cdot \log_2 n$
Busca	Mais rápida (árvore mais rasa)	Ligeiramente mais lenta
Rotações na inserção	No máximo 1	No máximo 2
Rotações na remoção	Até O(log n)	No máximo 3
Melhor para	Cargas dominadas por **leitura**	Cargas com muita **escrita**

Uma coleção de uso geral, como as do `java.util`, não sabe de antemão qual será o perfil de uso. A rubro-negra é a escolha mais equilibrada: paga um pouco a mais em cada busca (a árvore pode ser até 40% mais alta que a AVL) para garantir um número **constante** de rotações também na remoção, em vez dos O(log n) rebalanceamentos que a AVL pode precisar. Se a aplicação faz muito mais buscas que modificações, a AVL seria a melhor escolha — e é por isso que ela ainda é ensinada e usada em índices e estruturas especializadas.

6.5 Outras árvores binárias na biblioteca padrão

Além do `TreeMap`/`TreeSet`, vale registrar mais duas ocorrências:

`java.util.PriorityQueue` **— heap binário.** A documentação a descreve como "*an unbounded priority queue based on a priority heap*". Um heap binário é uma árvore binária **completa**, representada em um vetor em vez de nós encadeados (os filhos do índice *i* ficam em 2i+1 e 2i+2). Não é uma árvore de **busca**: ela só garante que cada nó é menor que seus filhos, o que basta para achar o mínimo, mas não para localizar um elemento qualquer. Por isso a própria documentação registra custos diferentes: O(log n) para `offer`, `add`, `poll` e `remove()`; **tempo linear** para `contains(Object)` e `remove(Object)`; e tempo constante para `peek` e `size`.

`java.util.HashMap` **— árvores rubro-negras dentro das listas de colisão.** Desde o Java 8, quando muitas chaves caem no mesmo *bucket*, a lista encadeada daquele bucket é convertida em uma árvore rubro-negra. A documentação pública registra apenas o efeito — "*when keys are Comparable, this class may use comparison order among keys to help break ties*" —, mas os comentários de implementação do OpenJDK são explícitos: a conversão ocorre a partir de `TREEIFY_THRESHOLD = 8` nós no bucket (desde que a tabela já tenha ao menos `MIN_TREEIFY_CAPACITY = 64` posições; caso contrário a tabela é redimensionada), e a árvore volta a ser uma lista simples quando o bucket encolhe para `UNTREEIFY_THRESHOLD = 6` nós.

Esse detalhe é uma ilustração perfeita do que medimos neste trabalho: o `HashMap` é O(1) no caso médio, mas no pior caso — muitas colisões — degenerava para O(n), porque o bucket era uma **lista encadeada**. A solução adotada foi exatamente trocar a lista por uma **árvore balanceada**, transformando o pior caso de O(n) em O(log n). É a mesma passagem que fizemos do Trabalho 1 para este.

6.6 Comparando com a nossa biblioteca

	Nossa `ArvoreBinaria`	**Nossa** `ArvoreAVL`	`TreeMap` **/** `TreeSet`
Estrutura	ABB sem balanceamento	ABB com balanceamento AVL	ABB com balanceamento rubro-negro
Altura garantida	Nenhuma (até n–1)	$\approx 1{,}44 \cdot \log_2 n$	$\approx 2 \cdot \log_2 n$
Busca / inserção / remoção	O(n) no pior caso	O(log n) garantido	O(log n) garantido
Comparação	`Comparator` no construtor	`Comparator` no construtor	`Comparable` ou `Comparator`

Ou seja: a estrutura que a biblioteca padrão de Java oferece para dados ordenados é conceitualmente a mesma que implementamos na seção 4, com um critério de equilíbrio diferente e uma troca explícita entre custo de busca e custo de modificação.

Fontes

Documentação oficial do Java 21 (Oracle):

• `java.util.TreeMap` — docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html

• `java.util.TreeSet` — docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeSet.html

• `java.util.PriorityQueue` — docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html

• `java.util.HashMap` — docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html

Os detalhes da *treeification* do `HashMap` estão nos comentários de implementação do código-fonte do OpenJDK, em `src/java.base/share/classes/java/util/HashMap.java`.

Material da disciplina: slides de árvores binárias e de árvores AVL (APNP) e os exemplos em C disponibilizados pelo professor (`arvBin.c`); classe `ArvoreBinariaBase` e interface `IColecao` do repositório da disciplina.

Código e relatório deste trabalho: github.com/ryanlimap/trabalho_tpa